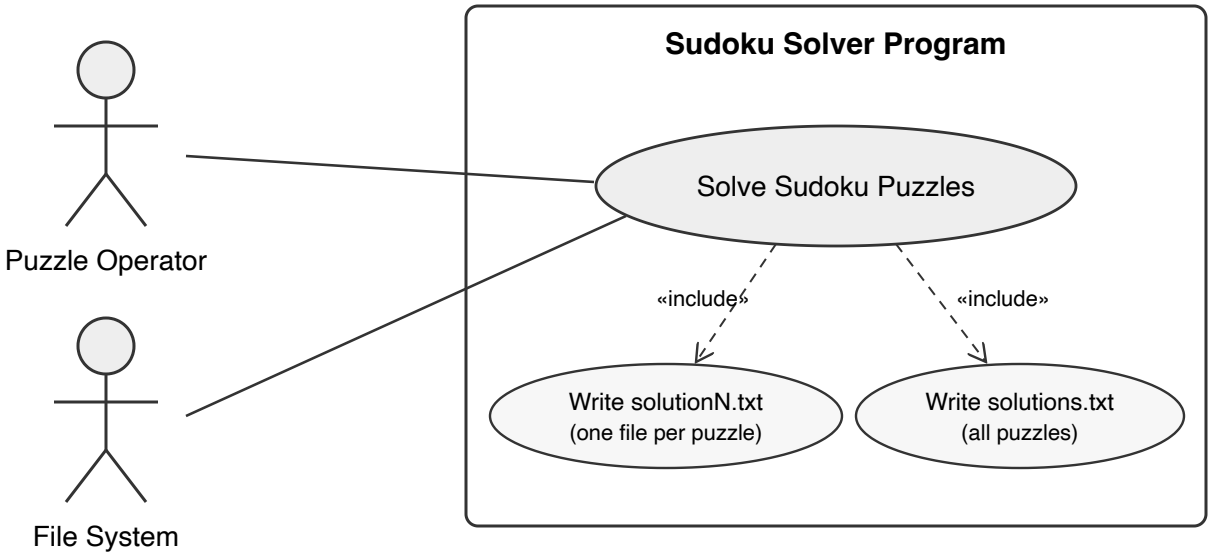
EECS 348 Assignment 6: UML Use Case

Francis Tryban · November 13, 2026 · Modified from Luke Tidball's Assignment 5 use case (Assignment5UseCase.pdf).
Modified with Claude Opus 5.5 (Anthropic, through Claude Code) to match Assignment6.cpp, which now writes a single solutions.txt as well as solution1.txt–solution5.txt.

a. Identify/discover a use case

- **Use case:** Solve Sudoku Puzzles
- **Primary actor:** Puzzle Operator, the person who runs the program
- **Supporting actor:** File System, which provides the puzzle files to the program and receives the generated output files
- **Goal:** The operator wants every supplied Sudoku puzzle solved, with each puzzle's solutions (or a no-solution message) saved both in its own solutionN.txt file and together in one solutions.txt file.
- **Use case diagram:**



b. Brief description

This use case allows a Puzzle Operator to run the program, which reads each puzzle file, checks and solves each puzzle, and writes the results two ways: to that puzzle's own solutionN.txt file (kept from Assignment 5) and to a single solutions.txt file that holds every puzzle (added in Assignment 6). In solutions.txt, each puzzle starts with "Puzzle: <file name>" and is followed by its solutions and "Solutions found: n", or by "No solution found."

c. Step-by-step flow (outline)

1. The Puzzle Operator starts the program, with or without puzzle file names.
2. The program creates (or empties) solutions.txt in the File System.
3. The program selects the next puzzle file.

4. The program writes "Puzzle: <file name>" to solutions.txt.
5. The program requests the puzzle file from the File System, and the File System provides it.
6. The program checks that the file contains a valid puzzle.
7. The program checks the puzzle's starting clues for conflicts.
8. The program searches for every solution, solving the puzzle only once.
9. The program writes each solution and "Solutions found: n", or "No solution found.", to solutionN.txt.
10. The program writes the same results to solutions.txt.
11. The program repeats steps 3–10 for the remaining puzzle files.
12. The program finishes after processing all the files.

d. Detailed description

Use case: Solve Sudoku Puzzles

Actors: Puzzle Operator (primary), File System (supporting).

Preconditions

- The Puzzle Operator can start the program.
- The puzzle files (puzzle1.txt–puzzle5.txt by default, or the names given on the command line) are in the current folder.
- The program is allowed to create files in the current folder.

Basic flow

1. **Operator starts the program.** The Puzzle Operator runs the Sudoku Solver Program. If file names are given, those files are solved in that order; otherwise puzzle1.txt–puzzle5.txt are used.
2. **Program prepares the combined output.** The program creates solutions.txt (or empties an old one) once, before any puzzle is solved, so every puzzle's results go into the same file.
3. **Program selects a puzzle file.** The program takes the next file from its list and writes the line "Puzzle: <file name>" (for example, "Puzzle: puzzle1.txt") to solutions.txt. It also opens that puzzle's own output file, solutionN.txt, where N is the number in the file name.
4. **Program reads the file.** The program requests the selected file from the File System and reads the puzzle's cells.
5. **Program validates the file format.** The program checks that the file has nine rows of nine cells and that each cell is either a digit from 1 through 9 or an underscore representing an empty cell.
6. **Program checks the starting clues.** The program checks that the puzzle's given numbers do not break the row, column or 3×3 box rules.
7. **Program searches for solutions.** If the clues are valid, the program uses recursive backtracking to find every valid way to complete the puzzle. The results are held in memory so the puzzle is solved only once.
8. **Program writes the result to solutionN.txt.** The program writes each solution in the same format as the input file, followed by "Solutions found: n". If there are none, it writes "No solution found."

9. **Program writes the result to solutions.txt.** The program appends the same solutions and count (or "No solution found.") under that puzzle's "Puzzle:" line.
10. **Program processes the next file.** The program repeats steps 3–9 for the next puzzle in the list.
11. **Program completes the run.** After all files are processed, the program closes its files and exits with status 0.

Alternative courses of action

- **A1. A puzzle file cannot be opened.** The program writes "Unable to read puzzle" to that puzzle's solutionN.txt, prints an error message on the screen, writes "No solution found." under the puzzle's "Puzzle:" line in solutions.txt, and continues with the next file.
- **A2. A puzzle file has missing, extra, or invalid cells.** Handled the same way as A1.
- **A3. The starting clues conflict.** The program writes "No solution found." for that puzzle to both solutionN.txt and solutions.txt, and continues with the next file.
- **A4. The puzzle has no valid completion.** The program writes "No solution found." to both files and continues with the next file.
- **A5. The puzzle has more than one valid completion.** The program writes every solution it finds and the total to both files, and continues with the next file.
- **A6. solutions.txt cannot be created.** The program prints an error message and stops with a nonzero exit status before solving any puzzle.
- **A7. A solutionN.txt file cannot be created.** The program prints an error message, writes "No solution found." for that puzzle in solutions.txt, and continues with the next file.

Postconditions

- solutions.txt contains one "Puzzle:" section for every file the program was given, in order.
- Every puzzle the program could read has a matching solutionN.txt file.